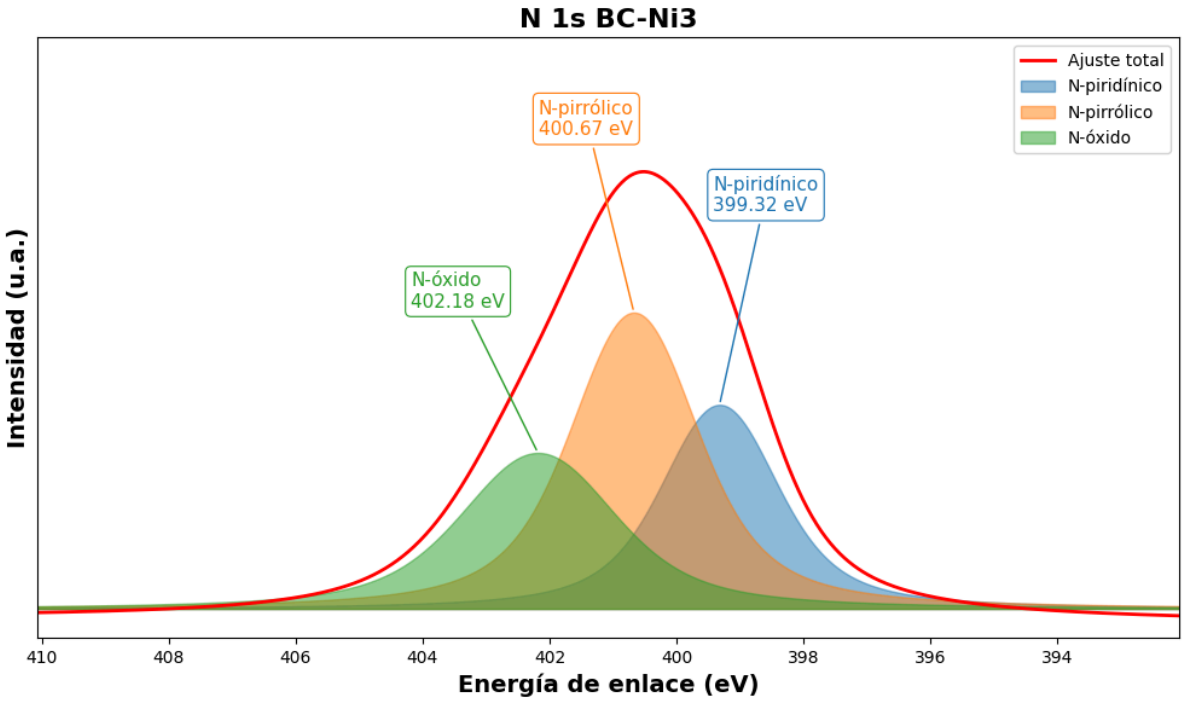
In [4]:
```
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(392.08, 410.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [399.8, 401.1, 402.5]
peak_intensities = [1600, 1200, 500]
peak_widths = [1.3, 1.4, 1.6]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo


# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['N-piridínico', 'N-pirrólico', 'N-óxido']
```

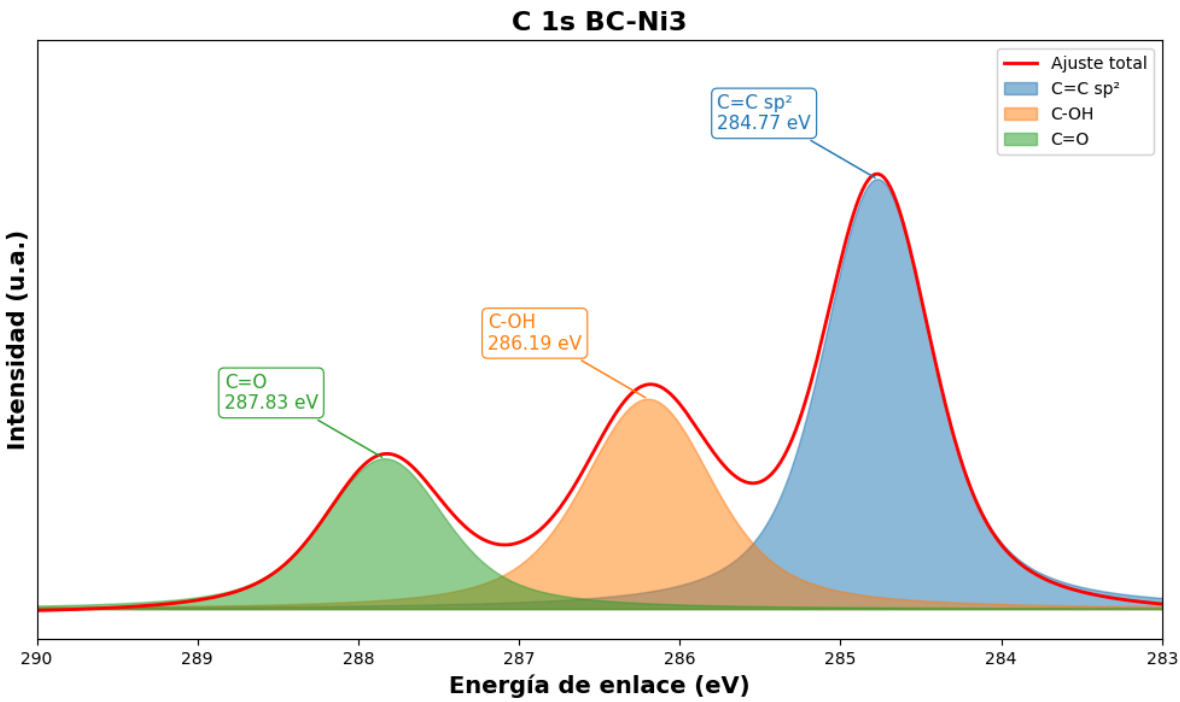
```
# Desplazamientos personalizados por etiqueta
x_offsets = [0.1, 1.5, 2.0]    # desplazamiento en x para cada etiqueta
y_offsets = [1300, 1200, 1000]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )

# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(binding_energy.min(), binding_energy.max())
ax.set_title("N 1s BC-Ni3", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.2)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(279.08, 298.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [284.8, 286.2, 287.8,]
peak_intensities = [500, 300, 200]
peak_widths = [0.4, 0.4, 0.4]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo


# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['C=C sp²', 'C-OH', 'C=O']

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
```

```
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    # Etiqueta con línea conectada
    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",  # Etiqueta con nombre y energía
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 1.0, peak_max + 80),  # Desplazar la etiqueta hacia afue
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(283, 290)
ax.set_title("C 1s BC-Ni3", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
# Extender el eje y para dejar espacio arriba
plt.ylim(top=intensity.max() * 1.0)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid
```

```
# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(279.08, 298.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [284.8, 286.2, 287.8,]
peak_intensities = [500, 300, 200]
peak_widths = [0.4, 0.4, 0.4]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo


# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['C=C sp²', 'C-OH', 'C=O']

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)
```

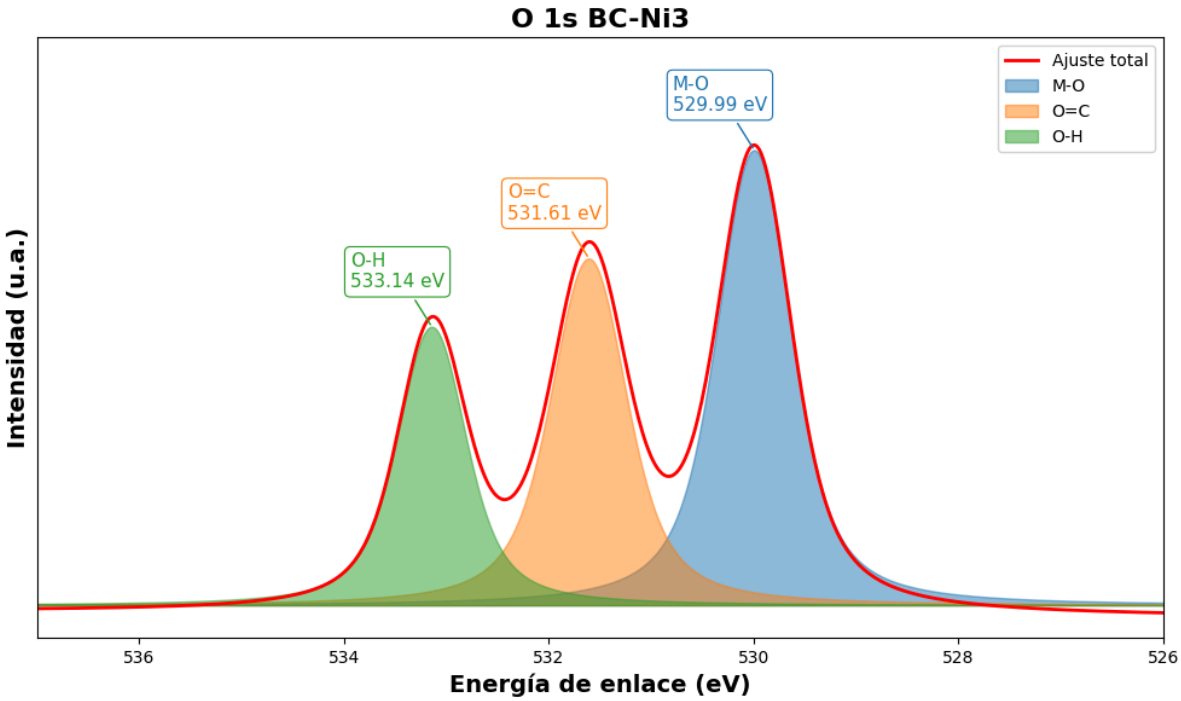
```
    # Etiqueta con línea conectada
    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",  # Etiqueta con nombre y energía
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 1.0, peak_max + 80),  # Desplazar la etiqueta hacia afue
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(280, 290)
ax.set_title("C 1s BC-Ni3", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
# Extender el eje y para dejar espacio arriba
plt.ylim(top=df["Counts/s"].max() * 0.05)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```

In [15]:

```
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(525.08, 545.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [530.0, 531.6, 533.1,]
peak_intensities = [1000, 800, 600]
peak_widths = [0.4, 0.4, 0.4]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley
```

```
background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['M-O', 'O=C', 'O-H']

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    # Etiqueta con línea conectada
    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",  # Etiqueta con nombre y energía
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 0.8, peak_max + 120),  # Desplazar la etiqueta hacia afu
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(526, 537)
ax.set_title("O 1s BC-Ni3", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.1)
ax.grid(alpha=0.3)
ax.grid(False)
```

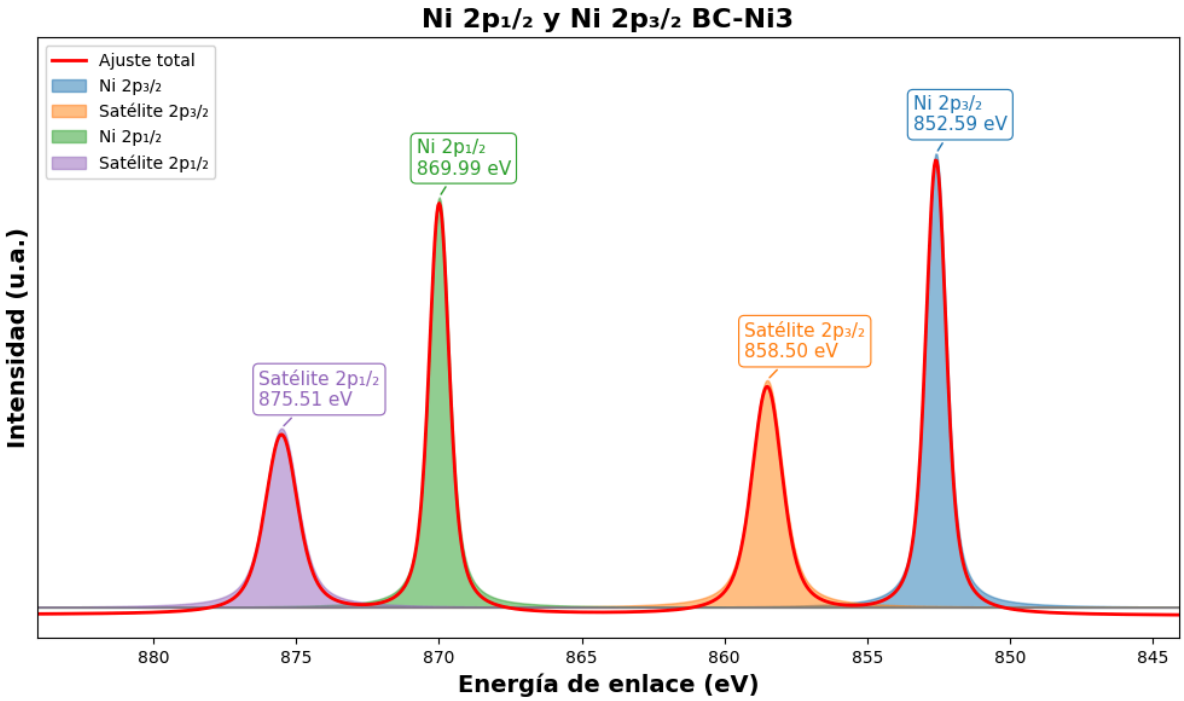
```
plt.tight_layout()
plt.show()
```



In [407…

```
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(844.08, 884.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [852.6, 858.5, 870.0, 875.5]
peak_intensities = [2000, 1000, 1800, 800]
peak_widths = [0.4, 0.6, 0.4, 0.6]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
     peak_intensities[3] * np.exp(-((binding_energy - peak_positions[3]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)
```

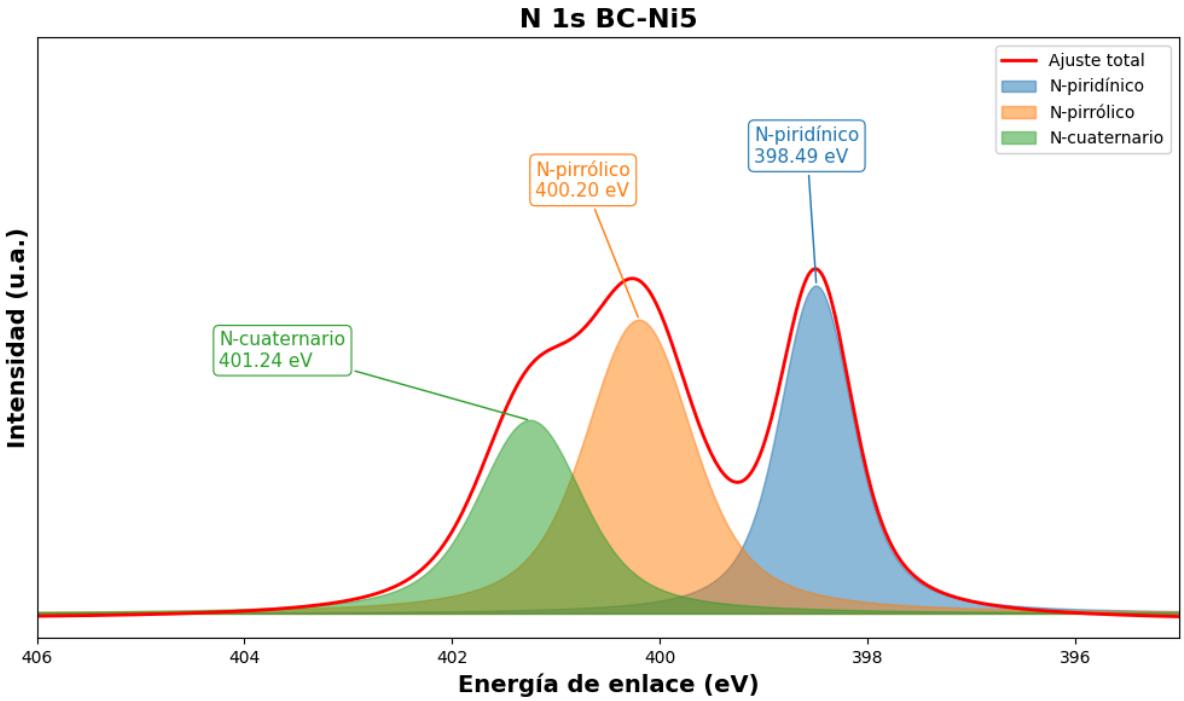
```
for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple' ]
labels = ['Ni 2p₃/₂', 'Satélite 2p₃/₂', 'Ni 2p₁/₂', 'Satélite 2p₁/₂']

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    # Etiqueta con línea conectada
    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",  # Etiqueta con nombre y energía
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 0.8, peak_max + 180),  # Desplazar la etiqueta hacia afu
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(binding_energy.min(), binding_energy.max())
ax.set_title("Ni 2p₁/₂ y Ni 2p₃/₂ BC-Ni3", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=df["Counts/s"].max() * 0.2)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [28]:

```
# BC-Ni5 NITRÓGENO

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(392.08, 410.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [398.5, 400.1, 401.2]
peak_intensities = [2400, 2000, 1600]
peak_widths = [0.4, 0.5, 0.6]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
```

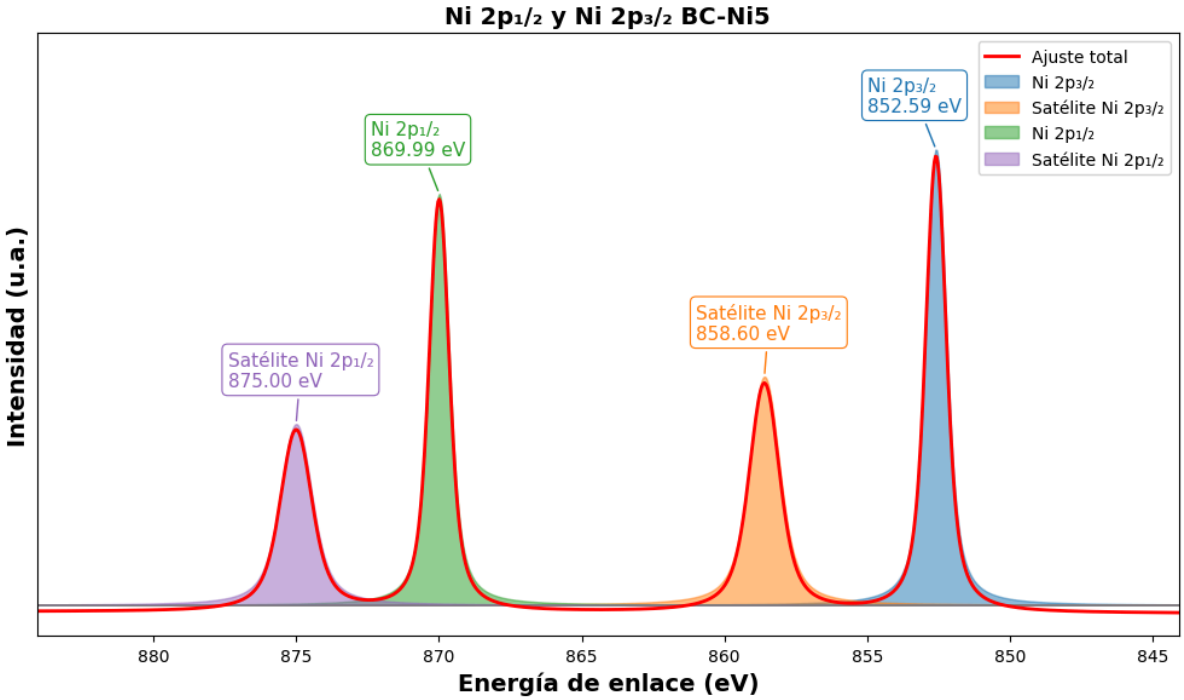
```
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['N-piridínico', 'N-pirrólico', 'N-cuaternario']

# Desplazamientos personalizados por etiqueta
x_offsets = [0.6, 1.0, 3.0]    # desplazamiento en x para cada etiqueta
y_offsets = [1000, 1000, 500]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(395, 406)
ax.set_title("N 1s BC-Ni5", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.6)
plt.grid(False)
ax.get_ylim()
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [411…

```
# BC-Ni5 NÍQUEL 2p3/2 Y 2p1/2

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(844.08, 884.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [852.6, 858.6, 870.0, 875.0]
peak_intensities = [2000, 1000, 1800, 800]
peak_widths = [0.4, 0.6, 0.4, 0.6]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
     peak_intensities[3] * np.exp(-((binding_energy - peak_positions[3]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
```

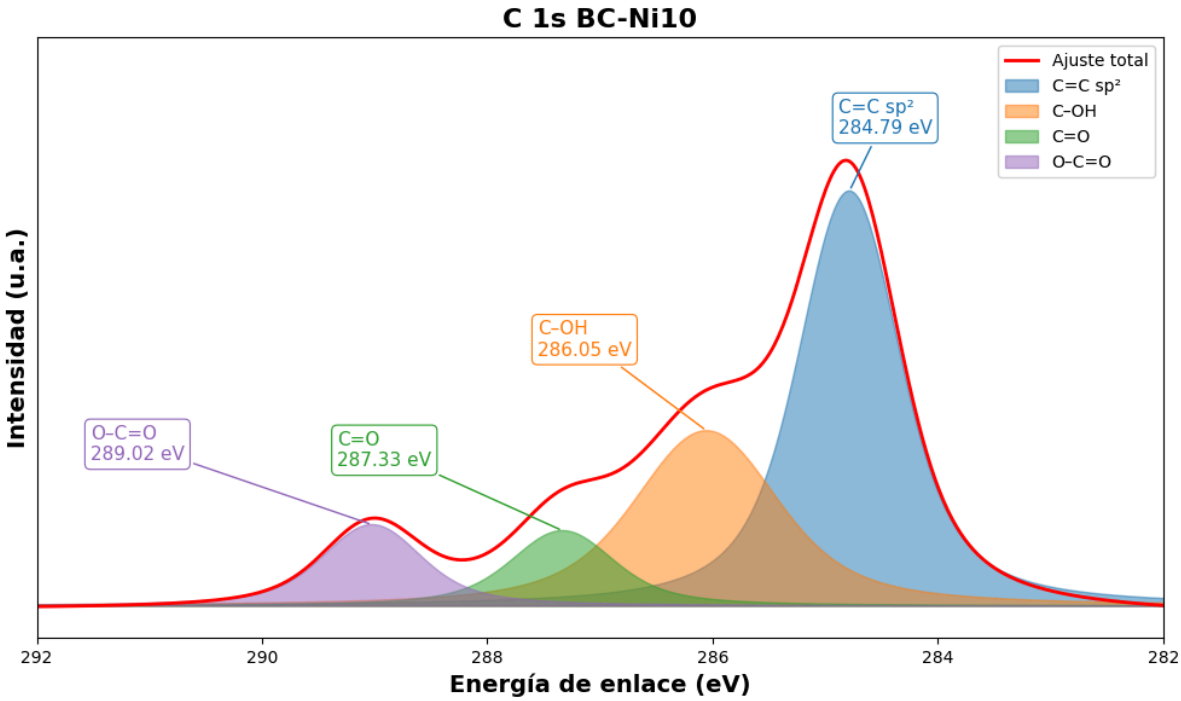
```
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple' ]
labels = ['Ni 2p₃/₂', 'Satélite Ni 2p₃/₂', 'Ni 2p₁/₂', 'Satélite Ni 2p₁/₂']

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    # Etiqueta con línea conectada
    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",  # Etiqueta con nombre y energía
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 2.4, peak_max + 250),  # Desplazar la etiqueta hacia afu
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(binding_energy.min(), binding_energy.max())
ax.set_title('Ni 2p₁/₂ y Ni 2p₃/₂ BC-Ni5', fontsize=14, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=df["Counts/s"].max() * 0.2)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [30]:

```
# BC-Ni10 CARBONO
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(279.08, 298.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [284.8, 286.1, 287.3, 289.0]
peak_intensities = [40000, 18000, 10000, 8000]
peak_widths = [0.5, 0.5, 0.5, 0.5]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
     peak_intensities[3] * np.exp(-((binding_energy - peak_positions[3]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
```

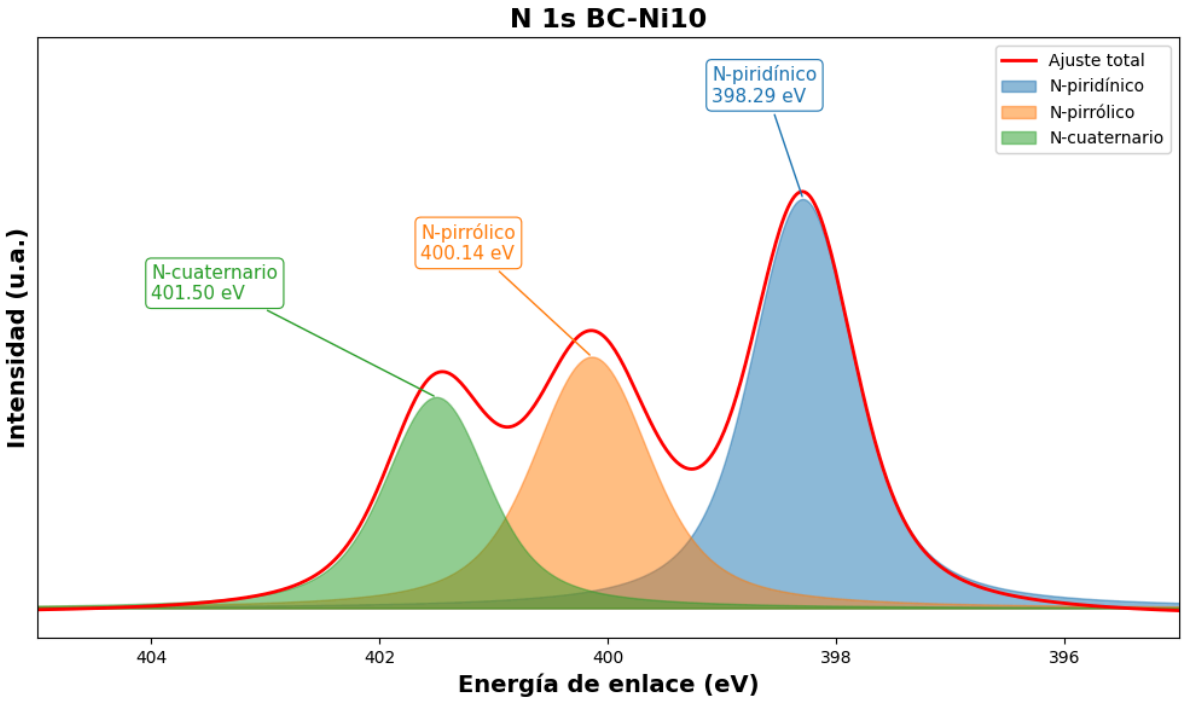
```
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple' ]
labels = ['C=C sp²', 'C-OH', 'C=O', 'O-C=O']

x_offsets = [0.1, 1.5, 2.0, 2.5]    # desplazamiento en x para cada etiqueta
y_offsets = [6800, 8500, 7500, 7500]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )

# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(282, 292)
ax.set_title("C 1s BC-Ni10", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
plt.ylim(top=intensity.max() * 1.3)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [35]:

```
# BC-Ni10 NITRÓGENO

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(392.08, 410.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [398.3, 400.1, 401.5]
peak_intensities = [1800, 1200, 1000]
peak_widths = [0.5, 0.5, 0.5]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
```

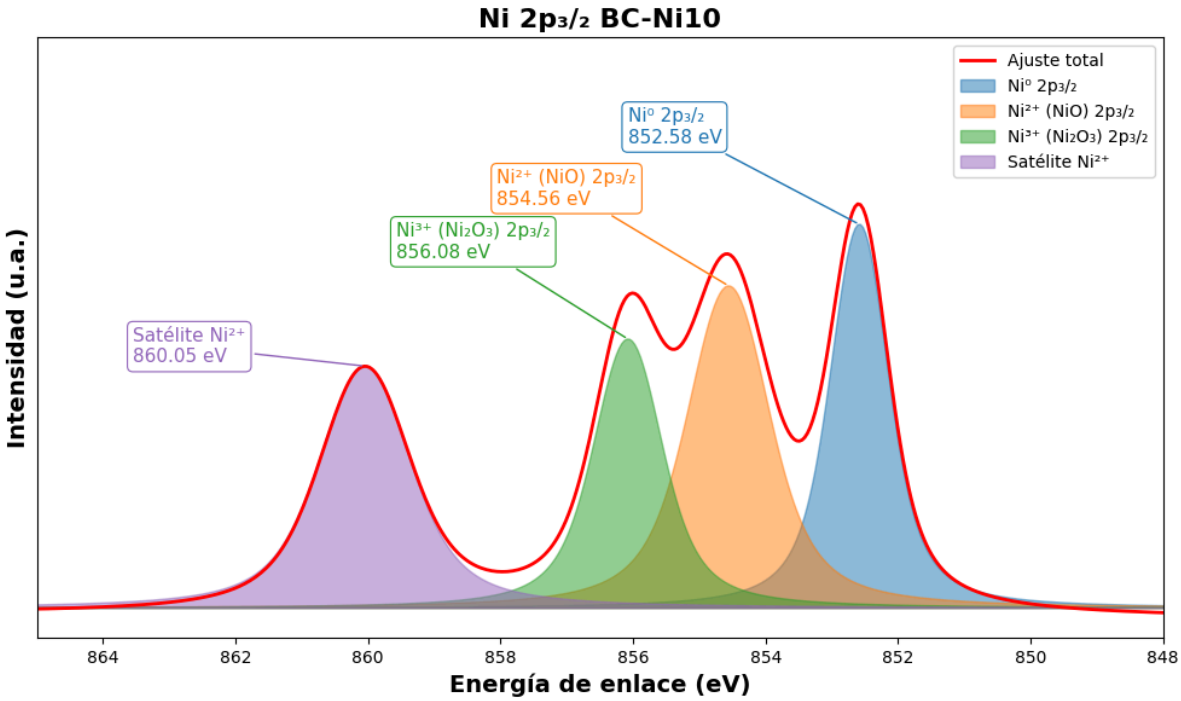
```
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['N-piridínico', 'N-pirrólico', 'N-cuaternario']

x_offsets = [0.8, 1.5, 2.5]    # desplazamiento en x para cada etiqueta
y_offsets = [500, 500, 500]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )

# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(395, 405)
ax.set_title("N 1s BC-Ni10", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.3)
plt.grid(False)
ax.get_ylim()
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [45]:

```
# BC-Ni10 NÍQUEL 2p1/2 y 2p3/2

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(844.08, 884.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [852.6, 854.5, 856.1, 860.0]
peak_intensities = [2000, 1800, 1500, 1200]
peak_widths = [0.5, 0.6, 0.6, 0.8]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
    peak_intensities[3] * np.exp(-((binding_energy - peak_positions[3]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
```

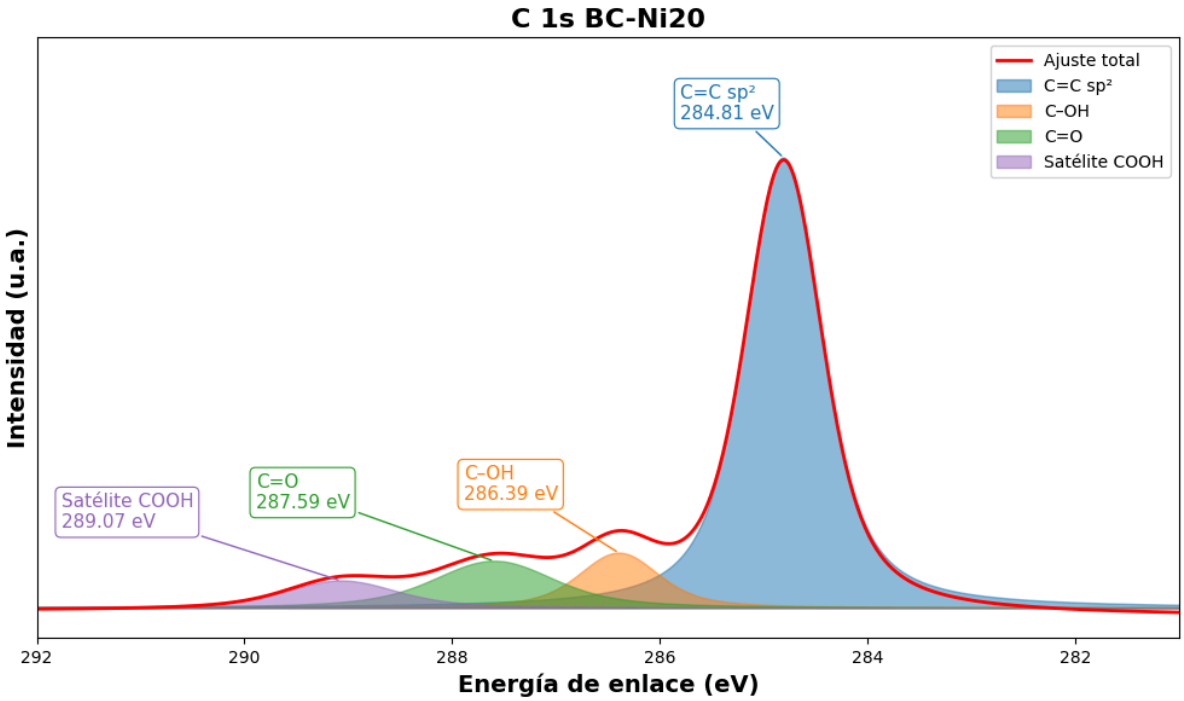
```
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple']
labels = ['Ni⁰ 2p₃/₂', 'Ni²⁺ (NiO) 2p₃/₂', 'Ni³⁺ (Ni₂O₃) 2p₃/₂', 'Satélite Ni²⁺']

x_offsets = [0.8, 4.5, 10.2, 10.2]    # desplazamiento en x para cada etiqueta
y_offsets = [500, 500, 500, 100]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + 3.5, peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )

# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(848, 865)
ax.set_title("Ni 2p₃/₂ BC-Ni10", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.4)
plt.grid(False)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [94]:

```
# BC-Ni20 CARBONO
import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(279.08, 298.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [284.8, 286.3, 287.6, 289.0]
peak_intensities = [70000, 12000, 8000, 5000]
peak_widths = [0.4, 0.5, 0.5, 0.6]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
     peak_intensities[3] * np.exp(-((binding_energy - peak_positions[3]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
```

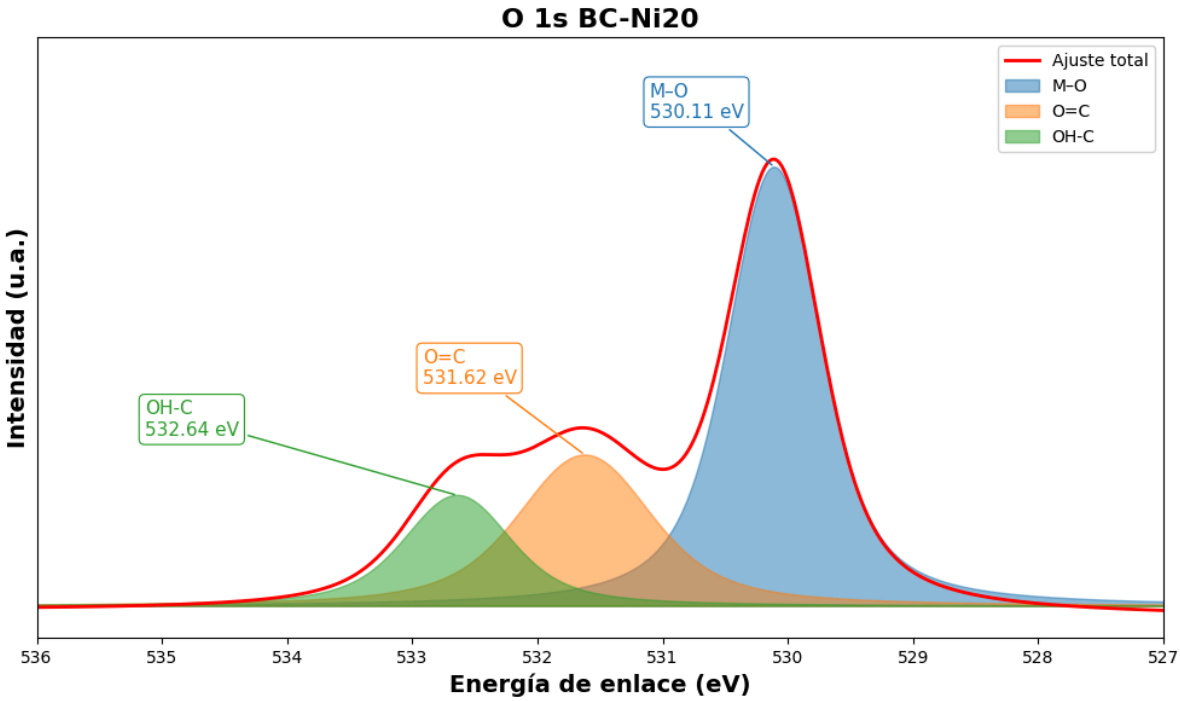
```
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple' ]
labels = ['C=C sp²', 'C-OH', 'C=O', 'Satélite COOH']

# Desplazamientos personalizados por etiqueta
x_offsets = [1.0, 1.5, 2.3, 2.7]    # desplazamiento en x para cada etiqueta
y_offsets = [8700, 11000, 11000, 11000]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(281, 292)
ax.set_title("C 1s BC-Ni20", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.ylim(top=intensity.max() * 1.3)
plt.grid(False)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [74]:

```
# BC-Ni20 OXÍGENO O1S

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid
import pandas as pd

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(525.08, 545.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [530.1, 531.5, 532.6]
peak_intensities = [40000, 15000, 12000]
peak_widths = [0.4, 0.5, 0.5]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
```

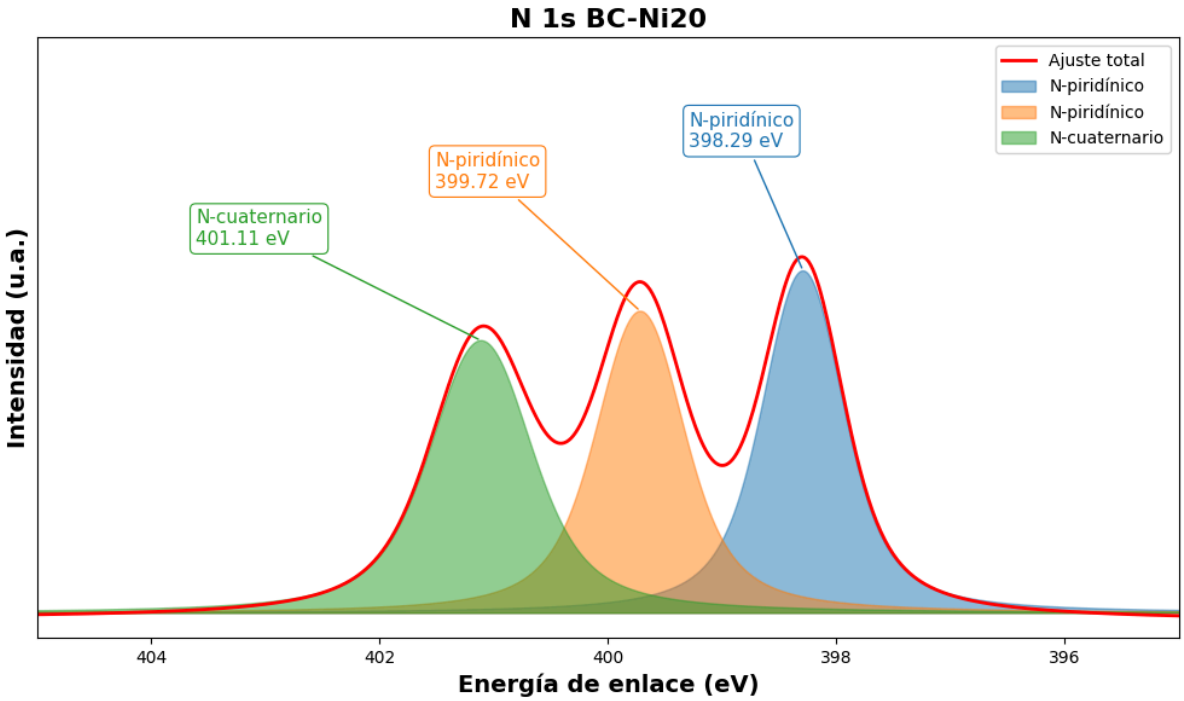
```
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['M-O', 'O=C', 'OH-C']

# Desplazamientos personalizados por etiqueta
x_offsets = [1.0, 1.3, 2.5]    # desplazamiento en x para cada etiqueta
y_offsets = [6000, 8000, 7000]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(527, 536)
ax.set_title("O 1s BC-Ni20", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
plt.ylim(top=intensity.max() * 1.3)
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [77]:

```
# BC-Ni20 NITRÓGENO

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(392.08, 410.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [398.3, 399.7, 401.1]
peak_intensities = [2500, 2300, 2000]
peak_widths = [0.4, 0.4, 0.5]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
    prefix = f'p{i}_'
```

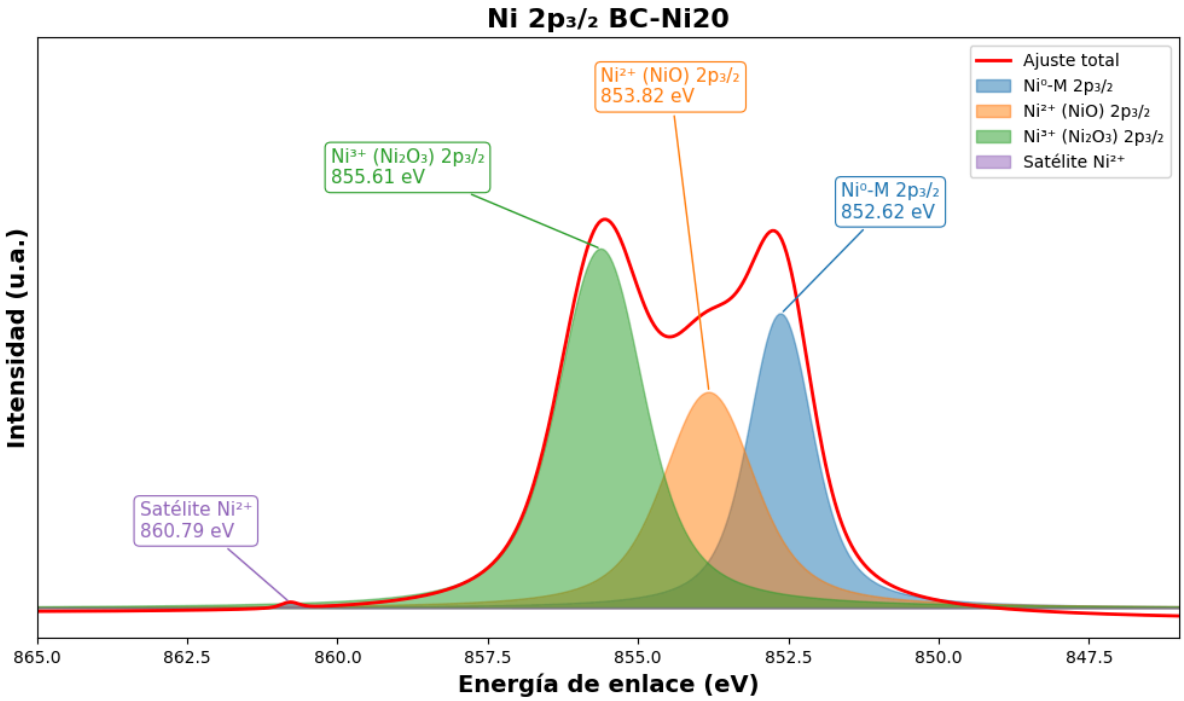
```
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green']
labels = ['N-piridínico', 'N-piridínico', 'N-cuaternario']

x_offsets = [1.0, 1.8, 2.5]    # desplazamiento en x para cada etiqueta
y_offsets = [1000, 1000, 800]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )



# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(395, 405)
ax.set_title("N 1s BC-Ni20", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
plt.ylim(top=intensity.max() * 1.6)
ax.get_ylim()
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```



In [80]:

```
# BC-Ni20 NÍQUEL

import numpy as np
import matplotlib.pyplot as plt
from lmfit.models import VoigtModel, PolynomialModel
from scipy.integrate import cumulative_trapezoid

# === 1. Simula o carga tus datos reales ===
# Simulación con ruido (sólo para ejemplo; reemplázalo con tus datos reales)
binding_energy = np.linspace(844.08, 884.08, 1000)
np.random.seed(0)
noise = np.random.normal(0, 100, binding_energy.size)

# Componentes centrales: piridínico, pirrólico, N-óxido (ligeramente desplazados de
peak_positions = [852.6, 853.8, 855.6, 860.6]
peak_intensities = [3000, 2200, 1800, 1500]
peak_widths = [0.6, 0.7, 0.8, 1.0]

true_signal = (
    peak_intensities[0] * np.exp(-((binding_energy - peak_positions[0]) ** 2) / (2
    peak_intensities[1] * np.exp(-((binding_energy - peak_positions[1]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
    peak_intensities[2] * np.exp(-((binding_energy - peak_positions[2]) ** 2) / (2
)
intensity = true_signal + noise

# === 2. Shirley background function ===
def shirley_background(x, y):
    """Computa fondo tipo Shirley para espectros XPS"""
    I0, If = y[0], y[-1]
    integral = cumulative_trapezoid(y[::-1] - If, x[::-1], initial=0)
    shirley = I0 - (I0 - If) * integral[::-1] / integral[-1]
    return shirley

background = shirley_background(binding_energy, intensity)

# === 3. Define modelos de Voigt + fondo polinómico para el ajuste ===
model = PolynomialModel(degree=1, prefix='bkg_')
params = model.guess(intensity - background, x=binding_energy)

for i, (pos, amp, wid) in enumerate(zip(peak_positions, peak_intensities, peak_widt
```

```
    prefix = f'p{i}_'
    voigt = VoigtModel(prefix=prefix)
    model += voigt
    params.update(voigt.make_params())
    params[f'{prefix}center'].set(value=pos, min=pos-0.5, max=pos+0.5)
    params[f'{prefix}amplitude'].set(value=amp, min=0)
    params[f'{prefix}sigma'].set(value=wid/2.35, min=0.1, max=2.0)  # Aproximación

# === 4. Ajuste ===
result = model.fit(intensity, params, x=binding_energy)

# === 5. Visualización ===
fig, ax = plt.subplots(figsize=(10, 6))

# Señal original y ajuste total

ax.plot(binding_energy, result.best_fit, label='Ajuste total', color='red', linewic

# Fondo

# Curvas individuales con etiquetas conectadas
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:purple']
labels = ['Ni⁰-M 2p₃/₂', 'Ni²⁺ (NiO) 2p₃/₂', 'Ni³⁺ (Ni₂O₃) 2p₃/₂', 'Satélite Ni²⁺']

x_offsets = [-1.0, 1.8, 4.5, 2.5]    # desplazamiento en x para cada etiqueta
y_offsets = [1100, 3000, 800, 800]  # desplazamiento en y para cada etiqueta

for i, (color, label_text) in enumerate(zip(colors, labels)):
    prefix = f'p{i}_'
    peak = VoigtModel(prefix=prefix).eval(params=result.params, x=binding_energy)
    ax.fill_between(binding_energy, peak, alpha=0.5, color=color, label=label_text)

    peak_pos = result.params[f'{prefix}center'].value
    peak_max = np.max(peak)

    ax.annotate(
        f"{label_text}\n{peak_pos:.2f} eV",
        xy=(peak_pos, peak_max),
        xytext=(peak_pos + x_offsets[i], peak_max + y_offsets[i]),
        arrowprops=dict(arrowstyle='-', color=color, lw=1),
        bbox=dict(boxstyle="round,pad=0.3", fc="white", ec=color, lw=0.8),
        fontsize=11,
        color=color,
        horizontalalignment='left',
        verticalalignment='center'
    )


# Estética
ax.set_xlabel("Energía de enlace (eV)", fontsize=14, weight='bold')
ax.set_ylabel("Intensidad (u.a.)", fontsize=14, weight='bold')
ax.set_xlim(846, 865)
ax.set_title("Ni 2p₃/₂ BC-Ni20", fontsize=16, weight='bold')
ax.legend()
ax.invert_xaxis()
plt.gca().tick_params(axis='y', which='both', left=False, right=False, labelleft=Fa
plt.grid(False)
plt.ylim(top=intensity.max() * 1.45)
ax.get_ylim()
ax.grid(alpha=0.3)
ax.grid(False)
plt.tight_layout()
plt.show()
```


Ni 2p3/2 BC-Ni20
Ni2+ (NiO) 2p3/2
853.82 eV
Ni3+ (Ni2O3) 2p3/2
855.61 eV
Ni0-M 2p3/2
852.62 eV
Satélite Ni2+
860.79 eV
Ajuste total
Ni0-M 2p3/2
Ni2+ (NiO) 2p3/2
Ni3+ (Ni2O3) 2p3/2
Satélite Ni2+
Intensidad (u.a.)
865.0
862.5
860.0
857.5
855.0
852.5
850.0
847.5
Energía de enlace (eV)

In [ ]: